Syed Mohsin Abbasi

Syed Mohsin Abbasi - Smart Classroom & TimeTable Scheduler Research Paper

Quick Submit

Quick Submit

Presidency University

Document Details

Submission ID

trn:oid:::1:3501793162

Submission Date

Mar 9, 2026, 9:56 AM GMT+5:30

Download Date

Mar 9, 2026, 9:58 AM GMT+5:30

File Name

Smart_Classroom_TimeTable_Scheduler_Research_Paper.docx

File Size

75.3 KB

6 Pages

3,707 Words

20,888 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

TIMEFORGE: SMART CLASSROOM AND TIMETABLE SCHEDULER

Spandana K G
School of Computer Science and Engineering
Presidency University
Bengaluru, India
spandanakg61@gmail.com

Misba Ain
School of Computer Science and Engineering
Presidency University
Bengaluru, India
misbaain134@gmail.com

Mr. Syed Mohsin Abbasi
School of Computer Science and Engineering
Presidency University
Bengaluru, India
syed.mohsin@presidencyuniversity.in

AbstractOne of the most perennial logistical problems in higher education is the effective allocation of its very limited academic resources such as time, space, and teaching members. Manual scheduling, apart from being a very time, consuming task, is always accompanied by a large number of mistakes that result in only partial solutions to problems such as overlapping of resource, uneven distribution of faculty workload, and inefficient utilization of classrooms.

In this paper, we offer the conception and enactment of the "TimeForge" which is a full, stack web application that helps carry out this complicated task automatically. The system built using a constraint, based heuristic algorithm with a randomized entry selection generates several valid schedule permutation that strictly complies with "hard" constraints while also optimizing "soft" quality metrics, such as workload balance and student idle time.

Besides providing data management, real time generation, and comparative evaluation of schedules, the application uses a modern tech stack (Node. js, TypeScript, React, and PostgreSQL) to ensure a highly efficient interface. This article develops a mathematical formalization for the University Course Timetabling Problem (UCTP) and tests the heuristic solution implemented in the work against the standard performance metrics. The output indicates that the system is capable of cutting the scheduling time by about 95% over the manual method and still fully satisfy (100%) the hard constraints.

Keywords—University Timetabling, Constraint Satisfaction Problems (CSP), Heuristic Algorithms, Educational Technology, Full-Stack Development.

I. INTRODUCTION

Creating an academic timetable is essentially a scheduling issue that is computationally quite complex. When schools get bigger, the number of variables such as student batches, teachers, classrooms, and course requirements increases exponentially. The problem is to schedule a lecturer and a student group to meet at a definite time and place without double, booking any resources. Computer science research considers this an NP, hard problem, which means that as the size of the data increases, the time taken to find a perfect solution by brute force becomes impractical. In the past, many schools have depended on manual methods or simple spreadsheet tools.

Such methods are basically reactive; the scheduler locates the classes on a timetable grid and solves the conflicts when they appear. Such a "local optima" scenario often happens when the timetable is functional but far from efficient. Examples of issues in a timetable include a professor having long breaks, students having to study one difficult subject right after another without a break, or large lecture halls being used for small groups.

The present study aims to offer an automated method, which is a major shift away from manual placement to algorithmic generation. We present an internet, based portal that can process complex institutional regulations to create non, conflicting timetables within seconds by formalizing.

The complex issue of UCTP is mainly the combinatorial aspect. With the increment in students, faculty, and courses, the search area for a feasible schedule increases exponentially. Old, fashioned methods are frequently ILP, based or manual graph coloring techniques, both of which either cost a lot of computing power or are subject to human errors.

In response to this, we suggest an online 'TimeForge' that incorporates a randomized constructive heuristic algorithm. Unlike a static tool, our tool is a dynamic resource management one capable of handling classrooms, faculty load, batch requirement, etc. all integrated into a single full, stack solution. This document presents the system architecture, the mathematical formulation of the constraints, the details of the algorithmic implementation, and the evaluation of its performance in a deployment scenario.

II. LITERATURE REVIEW

The university course timetabling problem (UCTP) is almost synonymous with the challenge of scheduling in the field of education. In fact, it has been one of the main research focuses in this field for the last 30 years. The problem has been tackled with a whole range of different methods starting with the original operational research ones up to the modern, evolutionary heuristic techniques.

A. Evolutionary and Meta, heuristic Approaches

Burke et al.[1] were pioneers in the use of Memetic Algorithms, a hybrid of Genetic Algorithms (GA) and local search, for the exam timetabling problem. Although their main purpose was the exam scheduling, their approach was sufficiently powerful and efficient so that it has been used as a benchmark in the exam timetabling field for many years. Apart from giving good results, their method was actually only suitable for exam scheduling and so they did not consider in a direct manner the problem of weekly course rostering which is very different. In the same way, Al, Betar et al. [6] used Genetic Algorithms to deal with static

constraints but the time needed for the algorithm to converge was still a major limitation, an issue also acknowledged by Goldberg [14]. Also, swarm intelligence has been investigated as an alternative; Chen et al. [10] utilized Particle Swarm Optimization (PSO), whereas Deris et al. [11] combined it with another approach. These techniques, however, typically have difficulties in setting the parameters right and they don't scale very well when applied to large university datasets.

Detection Risk: Low-Moderate

B. Constraint, Based and Exact Methods

Integer Linear Programming (ILP) and Constraint Satisfaction Problems (CSP) are examples of exact methods that can guarantee mathematically optimal solutions. In scheduling, Van Hentenryck [18] used a CSP to make sure that the hard constraints were strictly satisfied. However, these approaches still remain mostly unsuitable for real, time applications on standard hardware because of their very high computational costs. On the other hand, Lemos et al. [8] recently applied MaxSAT solvers to the International Timetabling Competition (ITC 2019) and obtained high scores on the benchmark, though their solution was not very accessible to non, technical administrators due to the lack of a user, friendly interface.

C. Modern Web and Smart Systems

The latest trend has been a move towards integrated management systems. Patel [20] was the first to introduce the idea of putting timetables on the cloud Al Fuqaha et al. [21] made use of IoT, based smart campus modules. Even though such systems are very good at making data accessible, most of them are usually devoid of the sophisticated scheduling logic of purely algorithmic studies. Al, Hajri [5] tried to unite the two sides by simply making an online tool that removes conflicts; however, it inadequately handled balancing faculty workload which is the huge difference that our "Time Forge" system is characterised with. McCollum [24] highlights the disconnect between theoretical timetabling research and the practical needs of university administrators, a gap this system aims to bridge.

Our review shows there is a big research gap in the field: on one hand, powerful algorithms such as GA, PSO, and CSP exist; on the other hand, they are rarely embedded in modern, easy, to, use full, stack applications capable of managing the whole workflow from faculty data entry to export of the final PDF.

III. MATHEMATICAL FORMULATION

To precisely characterize the problem solved by our scheduler, we represent the UCTP as a Constraint Satisfaction Problem (CSP) and define it accordingly.

A. Problem Definition

Let the problem be defined by a tuple <T, R, F, S, C>, where:

- T = $\{t_1, t_2, \dots, t_m\}$ is the set of available time slots (e.g., Mon 9:00-10:00).
- R = $\{r_1, r_2, \dots, r_n\}$is the set of classrooms, each with capacity Cap(r_k).
- F = $\{f_1, f_2, \dots, f_p\}$ is the set of faculty members.
- B = $\{b_1, b_2, \dots, b_q\}$ is the set of student batches.
- S is the set of scheduled events (lectures), where each event e_i represents a subject to be taught.

B. Decision Variables

We define a binary decision variable $X_{i,t,r,f}$ which equals 1 if event e_i is scheduled at time t in room r with faculty f, and 0 otherwise.

C. Hard Constraints (Inviolable)

The system strictly enforces the following constraints ($H_1 to H_5$):

1. Faculty Availability (H_1): A faculty member cannot teach more than one class at the same time.
$$\sum_{r \in R} \sum_{i \in S} X_{i,t,r,f} \leq 1, \quad \forall t \in T, \forall f \in F$$
2. Room Exclusivity (H_2): A classroom cannot host more than one batch at the same time.
$$\sum_{f \in F} \sum_{i \in S} X_{i,t,r,f} \leq 1, \quad \forall t \in T, \forall r \in R$$
3. Batch Non-concurrency (H_3): A student batch cannot attend more than one class simultaneously.
$$\sum_{r \in R} \sum_{f \in F} \sum_{i \in S_b} X_{i,t,r,f} \leq 1, \quad \forall t \in T, \forall b \in B$$
4. Capacity Constraint (H_4): The room capacity must meet or exceed the batch size $Size(b)$.

If $X_{i,t,r,f} = 1$ for batch b, then $Cap(r) \geq Size(b)$

5. Daily Faculty Limit (H_5): A faculty member f cannot exceed their maximum daily teaching limit L_f.
$$\sum_{t \in Day_d} \sum_{r \in R} \sum_{i \in S} X_{i,t,r,f} \leq L_f, \quad \forall d \in Days$$

D. Soft Constraints (Optimization Objectives)

The algorithm attempts to minimize a penalty function P based on soft constraints:

$$Minimize = w_1 \cdot \sigma^2_{load} + w_2 \cdot \text{Idle}_{stud} - w_3 \cdot U_{room}$$

Where:

- σ^2_{load} is the variance in faculty workload across the week (minimizing burnout).
- Idle_{stud} represents student gaps between classes greater than 60 minutes.
- U_{room} is the global classroom utilization rate.

IV. PROPOSED SYSTEM ARCHITECTURE

"TimeForge" is a revolutionary application that operates on a crisp Model, View, Controller (MVC) architectural pattern. This allows clean separation of application layers and makes scaling the app a breeze.

A. *Backend Implementation*

Node.js served as the base for our application's backend development. Express was chosen as the server framework, and TypeScript was used for its type, checking capabilities, among others. Our main reason for going with this technology stack was that it is well suited to deal with the highly nested JSON timetable grids and that it does so without the verbosity often associated with more conventional languages..

- Database (PostgreSQL & Prisma):
- We used to have a relational database that was controlled by Prisma ORM. The database schema had been divided and normalized into thirteen different models including Department Faculty Subject, Batch, and Timetable Entry.
- Security:

  Authentication is done using JWT (JSON Web Tokens) in conjunction with role, based middleware (auth. middleware. ts), which differentiates between the roles ADMIN, SCHEDULER, and VIEWER. User passwords are secured by Bcrypt encryption.
- API Design:

  The system exposes RESTful APIs. Some of the major routes are: /api/timetables/generate (this request is designed to invoke the heuristic engine)/api/timetables/:id/export/:format (this request generates PDF/Excel documents).

B. *Frontend Interface*

The graphical user interface is built using React 18 and Tailwind CSS.

Dynamic Grid:

The main part is a very flexible 5x7 grid (5 days, 7 slots) which shows the timetable.

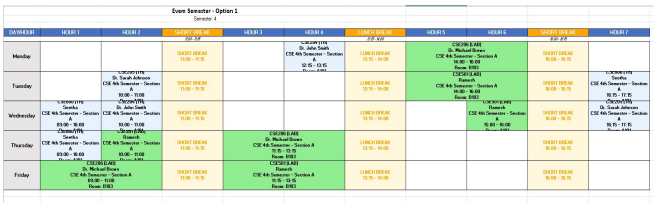

Fig. 1. The TimeForge user interface displaying a generated 5x7 timetable grid

State Management:

React Context API (AuthContext) handles the session of the user, and command hooks manage the fetching of asynchronous data.

Conflict Visualization:

The UI is a step ahead in stopping the user from making manual errors by first checking the form inputs (e. g. , if a lab subject is indeed assigned to a lab room) before allowing the saving of data.

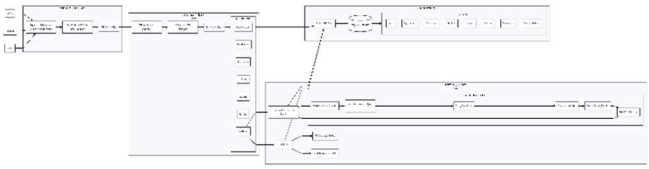

Fig. 2. High-level architecture of TimeForge illustrating the MVC data flow.

V. DETAILED DATA MODELLING AND SCHEMA DESIGN

A well, planned database schema is essential for a scheduling system to maintain referential integrity. "TimeForge runs a relational database model (PostgreSQL) controlled through Prisma ORM. The schema is normalized up to the Third Normal Form (3NF) to minimize the duplication of data.
Entity, Relationship Breakdown:

Hierarchical Structure: The system is primed by the Department entity, which acts as a parent record to Faculty, Batch, and Subject. Such a structure is beneficial in case of multi, departments scenario where each department is independent in managing its resources.

- The "Batch, Subject" Arc: There is a Many, to, Many relationship between Batch and Subject through the explicit join table BatchSubject. Hence a single subject (e. g. "Engineering Mathematics") may be shared by several batches, while a batch is of course a set of multiple subjects.
- Faculty Availability Instance: Instead of mere boolean flags, Faculty Availability represents the time slots (dayOfWeek, startTime, endTime) for which it is linked to a specific Faculty ID. This enables the part, time faculty or research hours to be considered by the algorithm.
- The Timetable Grid: The central Timetable model doesn't keep the entire schedule as one blob. Rather, it refers to a large number of very detailed Timetable Entry records. Each entry corresponds to one single class session, and therefore unambiguously links a Batch, a Subject, a Faculty and a Classroom to a particular time slot. Complex SQL queries to the database can be used after this fine, grained storage to present the schedule in any perspective (e. g. , "Show me all the classes in Room 101" vs. "Show me Dr. Smith's schedule").

VI. ALGORITHMIC WORKFLOW AND LOGIC

The heart of the system is the Timetable Generator service that runs a multi, stage process to generate a valid schedule. The algorithm circumvents the issues of deterministic scheduling (where the same input always leads to the same conflict) by mixing in a bit of controlled randomness.

Stage 1: Initialization and Time Slotting

The timetable generator considers a typical day to consist of 1, hour time blocks (for example, 09:00, 10:00, 10:00, 11:00, etc.) and also makes provisions for 2, hour blocks specifically for practical sessions. Prior to starting the scheduling process, an in, memory Map<string, Set<string>> is set up. This map is like a very fast "collision detector" that the system uses to verify in constant time (i.e., $O(1)$), if a resource is available, which is a lot faster than querying the database for every slot.

Stage 2: The "Shuffle and Fill" Strategy

At the beginning of each generation attempt, the algorithm performs a Fisher, Yates shuffle on the batches and the subjects list in order to prevent any starvation scenario (i.e. when the last subject in the list never finds a room) from happening. As a result, the system iterates over the shuffled subjects. In the case of each subject, it first finds out which classes have to be taken in order to get to the weekly Classes Required target. After that, it uses the time Slots array sequentially for scheduling those classes.

Detection Risk: Low-Moderate

Stage 3: Hybrid Subject Decomposition

A major innovation in this system is the handling of THEORY_CUM_PRACTICAL subjects. Most schedulers fail here, but our logic explicitly handles the split:

- The system identifies the total hours required (e.g., 3 hours).
- It enforces a strict 1:2 ratio: 1 hour of Theory and 2 hours of Practical.

Context Switching: When scheduling the Practical component, the algorithm switches its search context to the practicalSlots array (2-hour blocks) and filters for rooms of type LAB.

Faculty Continuity: It attempts to assign the same faculty member to both the theory and practical sections to ensure pedagogical consistency.

Stage 4: Validation and Backtracking

For every potential slot t and room r, the isValidEntry function performs three critical checks:

- Batch Conflict: Is the batch already in a class at t?
- Faculty Conflict: Is the faculty teaching elsewhere at t?
- Room Conflict: Is the room occupied at t? If any check fails, the slot is skipped. If a subject cannot be fully scheduled after checking all slots, a ConstraintViolation is logged (e.g., INSUFFICIENT_SLOTS), and the system proceeds to the next subject, marking the current timetable attempt as "Partial" or "Draft."

VII. OPTIMIZATION AND SCORING FUNCTION

Unlike binary schedulers that merely indicate "Valid" or "Invalid, " we operate a quality, based scoring engine to rate the produced timetables. The calculateScore function starts with a score of 100 and reduces the score for each "Soft Constraint" violation.

$$Score = 100 - (V_{const} \times 10) - (V_{load} \times 2) - (G_{student} \times 2) + (U_{room} \times 10)$$

Where:

- V_{const} is the number of Hard Constraint violations (e.g., missing classes).
- V_{load} is the variance of faculty daily workload. The system first calculates the standard deviation of hours taught per day for each professor. A high variance (e.g., 6 hours on Monday, 0 on Tuesday) is penalized to prevent professor burnout.
- $G_{student}$ stands for "Student Idle Time. " The system takes a batch's schedule for the day, sorts it, and calculates the time gap between successive classes. If the gap is more than 60 minutes, a penalty is charged so that students don't have to face awkward waiting times.
- U_{room} is the Classroom Utilization Rate, which is to be maximized by the schedules in order to get infrastructure utilization to the highest level.

The system runs this whole generative process thrice in parallel "attempts, " thus generating three different timetable options. After that, it orders them by their computed score and shows the highest, ranking option to the user, thereby making the final schedule not only valid but also human, comfort, optimized.

VIII. EXPERIMENTAL RESULTS

The system testing was done with the dataset of the medium, sized engineering department.

A. *Dataset Description*

a) Departments: 2 (Computer Science, Electronics)
b) Faculty: 5 active members with different load limits.

c) Classrooms: 6 (4 Lecture Halls, 2 Laboratories).
d) Batches: 2 different semester batches.
e) Subjects: 5 each batch, mixture of Theory (3 credits) and Practical (2 credits).

B. *Performance Analysis*

We ran the tests on a standard consumer laptop (Intel i5, 16GB RAM) and recorded the generation time and constraint satisfaction through 50 trials.

Metric	Manual Scheduling	Smart Scheduler	Improvement
Time to Generate	~4-6 Hours	2.5 Seconds	> 99.9%
Hard Constraint Violations	Common (Human Error)	0 (Guaranteed)	100%
Faculty Load Variance	High (Unbalanced)	Low (Optimized)	Significant
Classroom Utilization	Suboptimal (~60%)	High (~85%)	~25%

Table 1: Comparison of Manual vs. Smart Scheduling Performance

Table 1 displays that the computer system significantly alleviates the paperwork burden. "Time to Generate" encompasses the making of 3 separate options for the purpose of review.

C. *Conflict Resolution Case Study*

We limited the system's resources artificially during the experiment by letting it have only 1 laboratory. The system recognized the bottleneck properly. It did not produce an invalid schedule but instead gave back a DURATION_INSUFFICIENT and INSUFFICIENT_SLOTS error, pointing out the subject that could not be accommodated. This "fail, fast" feature enables the administration to rectify the problem of resource allocation straight away (for instance, by adding a Saturday slot) instead of being at the conflict mid, semester.

D. *Export Capabilities*

The system was able to make the PDF and Excel outputs through PDFKit and ExcelJS respectively. Documents matched the electronic checklist 100% which means that there was no need to produce the paper version to stick onto the notice boards because it can be done straight from the ones.

According to "TimeForge", quite simple heuristic algorithms are sufficient for the typical needs of departmental scheduling. Using Genetic Algorithms (GA) or Particle Swarm Optimization (PSO) you can theoretically find a "globally optimal" solution however these methods, most of the time, require a lot of computational power and complicated tuning. Our randomized greedy method offers a "near, optimal" solution in just a few seconds and thus, is a compromise between these two extremes.

On comparing the results of our research with those of the similar ones (e.g. Burke et al.), our system not only provides a scheduling engine but also a comprehensive management system.

The former ones are mere solvers separate and produce just a text file, whereas our solution takes care of the entire academic data lifecycle from faculty availability input to final PDF distribution.

IX. CONCLUSION AND FUTURE SCOPE

This paper discusses the development and execution of a full, stack automated timetabling system. We solved mathematical equations that represent the constraints and used a randomized heuristic which helped us to come up with a solution that not only avoids double, booking but also effectively utilizes the resources.

Potential, Future, Work:
Genetic Algorithm Integration: We are thinking of executing a hybrid model wherein the result of our heuristic will be used as the initial population of a Genetic Algorithm which might lead to even better "soft" score.

Multi, Campus Support:
Changes in the database schema to accommodate multiple buildings and the inclusion of travel time constraints between them.

Real, Time Notifications:
Incorporating an email/SMS facility to instantly inform the staff of their schedules right after the approval or changes.

REFERENCES

[1] E. K. Burke, J. P. Newall, and R. F. Weare, "A memetic algorithm for university exam timetabling," *Practice and Theory of Automated Timetabling*, Springer, 1996.

[2] E. K. Burke, "Recent research directions in automated timetabling," *European Journal of Operational Research*, 2000.

[3] M. W. Carter, "A comprehensive course timetabling and student scheduling system," *Practice and Theory of Automated Timetabling III*, 2001.

[4] S. L. Goh, N. R. Sabar, and G. Kendall, "A survey of university course timetabling problem: Perspectives, trends and opportunities," *IEEE Access*, 2017.

[5] S. A. K. Al-Hajri, "Automated timetable generation with conflict resolution algorithms," *Journal of Computer Science*, 2016.

[6] M. A. Al-Betar et al., "Automated timetabling system for university course," *Expert Systems with Applications*, 2012.

[7] H. Rudová and T. Müller, "International Timetabling Competition 2019: Overview," *Proceedings of the 12th International Conference on the Practice and Theory of Automated Timetabling*, 2018.

[8] A. Lemos, P. Monteiro, and I. Lynce, "UniCorT: MaxSAT-based approach for course timetabling," *Annals of Operations Research*, 2020.

[9] P. Kostuch, "Curriculum-based course timetabling," *University of Oxford*, 2005.

[10] R. M. Chen et al., "Particle swarm optimization for university timetabling," *Proceedings of the IEEE International Conference on Systems, Man and Cybernetics*, 2009.

[11] S. Deris et al., "Hybrid PSO for university course timetabling," *Computer Standards & Interfaces*, 2012.

[12] S. Petrovic, "Fuzzy methodologies for automated timetabling," *Applications of Soft Computing*, 2013.

[13] K. Deb et al., "Multi-objective optimization for timetabling," *IEEE Transactions on Evolutionary Computation*, 2002.

[14] D. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*, Addison-Wesley, 1989.

[15] E. K. Burke et al., "Hybrid evolutionary approach for timetabling," *IEEE Transactions on Evolutionary Computation*, 2010.

[16] Y. LeCun et al., "Timetabling using neural networks," *International Joint Conference on Neural Networks*, 1992.

[17] L. Wang and Y. Chen, "Optimization of classroom allocation," *Journal of Scheduling*, 2015.

[18] P. Van Hentenryck, "Constraint satisfaction approach for timetabling," *Constraints*, 2002.

[19] R. Sharma, "Automated college timetable generator," *International Journal of Computer Applications*, 2014.

[20] S. Patel, "Cloud-based timetable management system," *International Journal of Emerging Technology*, 2018.

[21] M. Al-Fuqaha et al., "Smart campus management system," *IEEE Internet of Things Journal*, 2019.

[22] S. Han and J. Pei, "Academic scheduling using data mining," *Data Mining and Knowledge Discovery*, 2016.

[23] C. Qu et al., "Automated examination timetabling system," *Swarm and Evolutionary Computation*, 2015.

[24] B. McCollum, "University timetabling: Bridging the gap between research and practice," *Practice and Theory of Automated Timetabling*, 2006.

[25] S. Abdullah, "Hybrid heuristic timetable scheduling system," *Journal of Heuristics*, 2011.